

Density estimation

Introduction

- Last week we looked at Bayes decision theory
- Nice theoretically, but assumes knowledge about probability density distribution.
- This is rarely available, so how do we proceed?
- Estimate densities from data!

Overview

- In this lecture we will cover:
 - Parametric density estimation
 - Maximum likelihood
 - Non-parametric density estimation
 - Histogram approach
 - Parzen window approach
 - k-nearest neighbor approach
 - k-nearest neighbor classification

Parametric density estimation

What do we mean by parametric?

- Assume that the *form* of the probability density function is known.
- Estimate *parameters* from data

☐ Show distribution names

Distribution 1

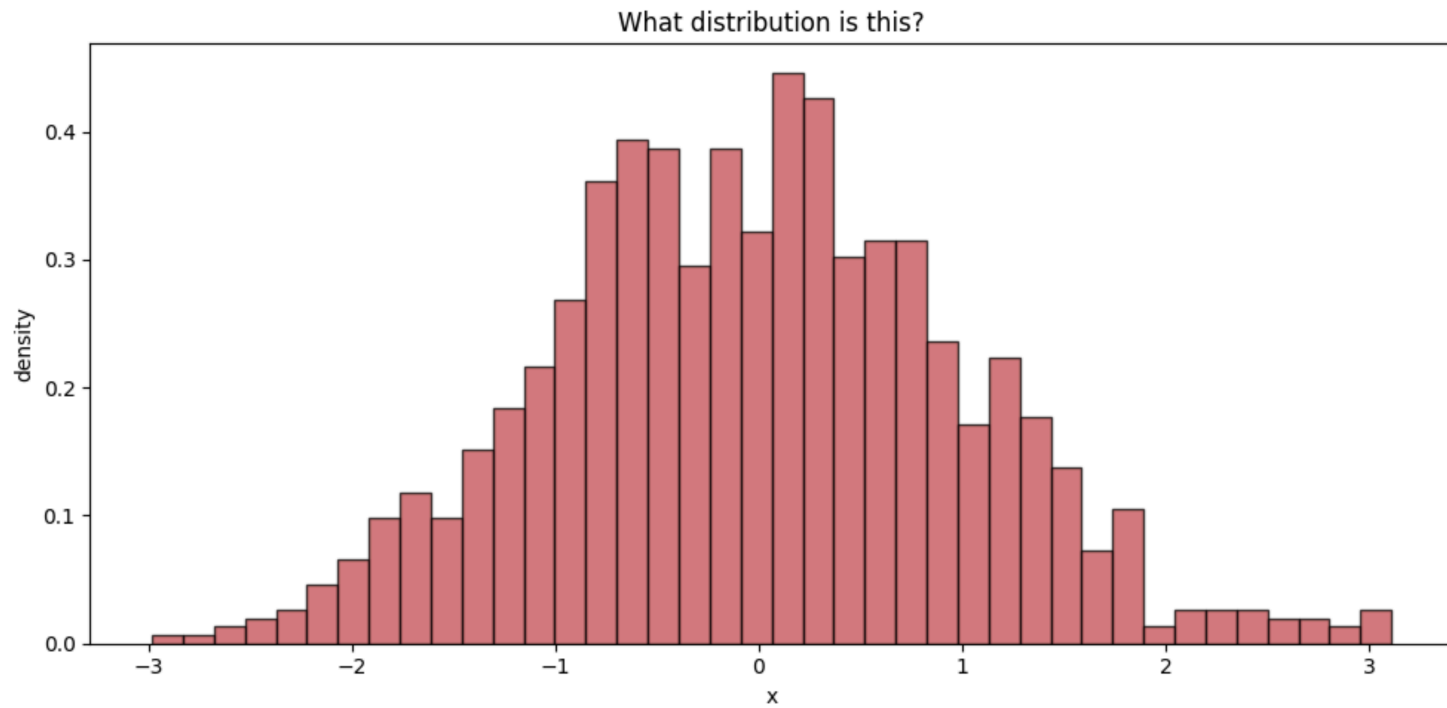
Distribution 2

Distribution 3

Distribution 4

Distribution 5

Distribution 1



Recall

- Obtain *realizations* from probability density function:
 - $p(\mathbf{x}_1, \boldsymbol{\theta}) \rightarrow \mathbf{x}_1$
 - $p(\mathbf{x}_5, \boldsymbol{\theta}) \rightarrow \mathbf{x}_5$
- Joint:
 - $p(\mathbf{x}_1, \mathbf{x}_5, \boldsymbol{\theta}) \rightarrow \mathbf{x}_1$ and $\mathbf{x}_5$
- If independent:
 - $p(\mathbf{x}_1, \mathbf{x}_5, \boldsymbol{\theta}) = p(\mathbf{x}_1, \boldsymbol{\theta})p(\mathbf{x}_5, \boldsymbol{\theta})$

Likelihood function

- Let $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and assume independent and identically distributed random variables (i.i.d.).
- Let the set of parameters for a particular distribution form a parameter vector $\boldsymbol{\theta}$.
- The likelihood function of $\boldsymbol{\theta}$ with respect to the random variable X can be expressed as:

$$\blacksquare p(X; \boldsymbol{\theta}) = \prod_{i=1}^N p(\mathbf{x}_i; \boldsymbol{\theta})$$

- Idea:
 - Find $\boldsymbol{\theta}$ that makes X most probable.
- Therefore:
 - $\hat{\boldsymbol{\theta}}_{ML} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} p(X; \boldsymbol{\theta})$

Log-likelihood

- In most cases, we work with the log-likelihood instead of the likelihood itself.
 - Simplifies the mathematics.
 - Can be more numerically stable if we have the product of many small probabilities.
- Log-likelihood:
 - $L(\boldsymbol{\theta}) = \log \prod_{i=1}^N p(\mathbf{x}_i; \boldsymbol{\theta})$
 - Remember: $\log(ab) = \log(a) + \log(b)$
- Want:
 - $\frac{\partial}{\partial \boldsymbol{\theta}} L(\boldsymbol{\theta}) = \mathbf{0}$

Example with univariate normal

- Let us consider the case of a univariate normal distribution
- Parameters:

- $\boldsymbol{\theta} = (\theta_1, \theta_2)^T = (\mu, \sigma^2)^T.$

- First: log-likelihood

- $\log \prod_{i=1}^N p(\mathbf{x}_i; \boldsymbol{\theta}) =$

- Then: derivative

- $\frac{\partial}{\partial \boldsymbol{\theta}} \log p(\mathbf{x}_i; \boldsymbol{\theta}) =$

- Lastly: setting to zero

- $\sum_{k=1}^N = \mathbf{0}$

Example with univariate normal continued

- Putting it all together gives us:

- For θ_1 :

- $\hat{\theta}_1 = \hat{\mu} = \frac{1}{N} \sum_{k=1}^N x_k$

- For θ_2 :

- $\hat{\theta}_2 = \hat{\sigma}^2 = \frac{1}{N} \sum_{k=1}^N (x_k - \mu)^2$

- Note: In many cases, we will use these estimators for mean and covariances regardless.

☐ Show log-likelihood

$\mu=0.0, \sigma=0.8$

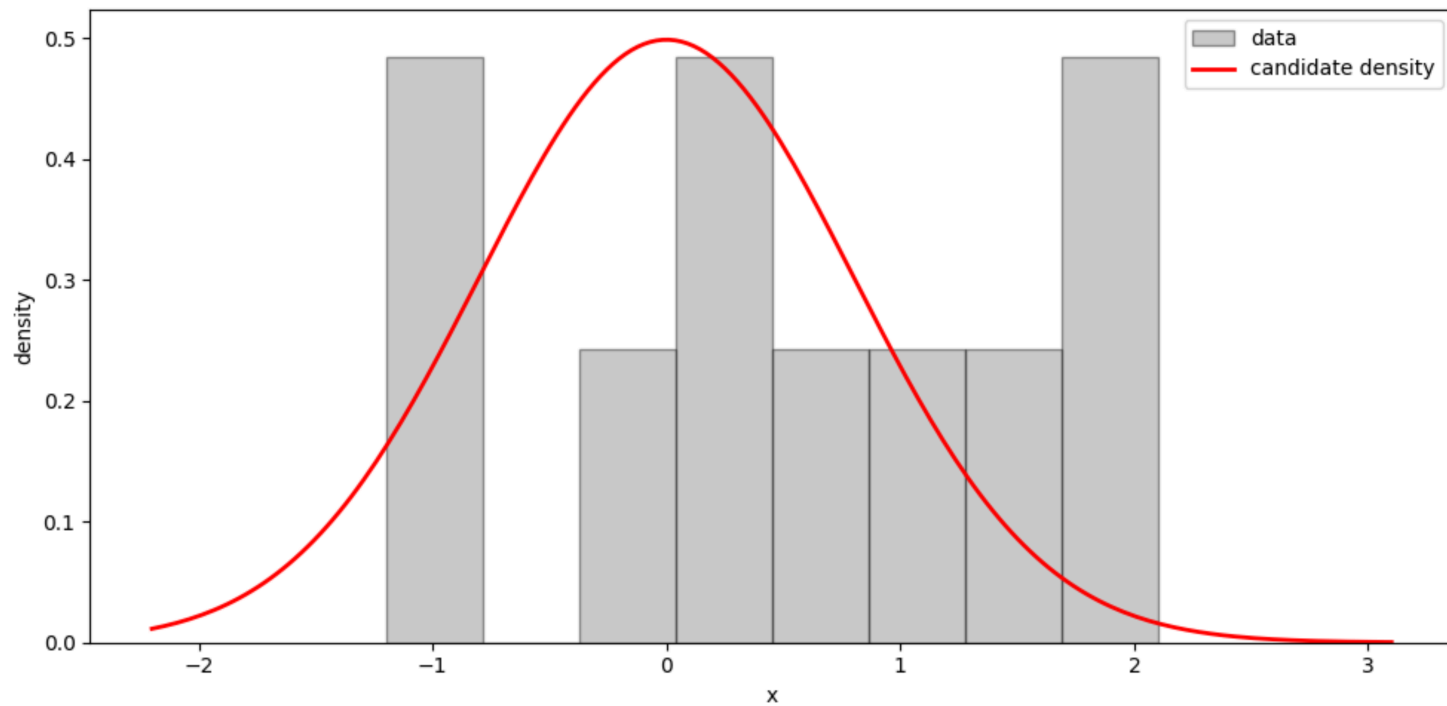
$\mu=0.5, \sigma=0.8$

$\mu=0.9, \sigma=0.8$

$\mu=0.9, \sigma=0.4$

$\mu=0.9, \sigma=1.5$

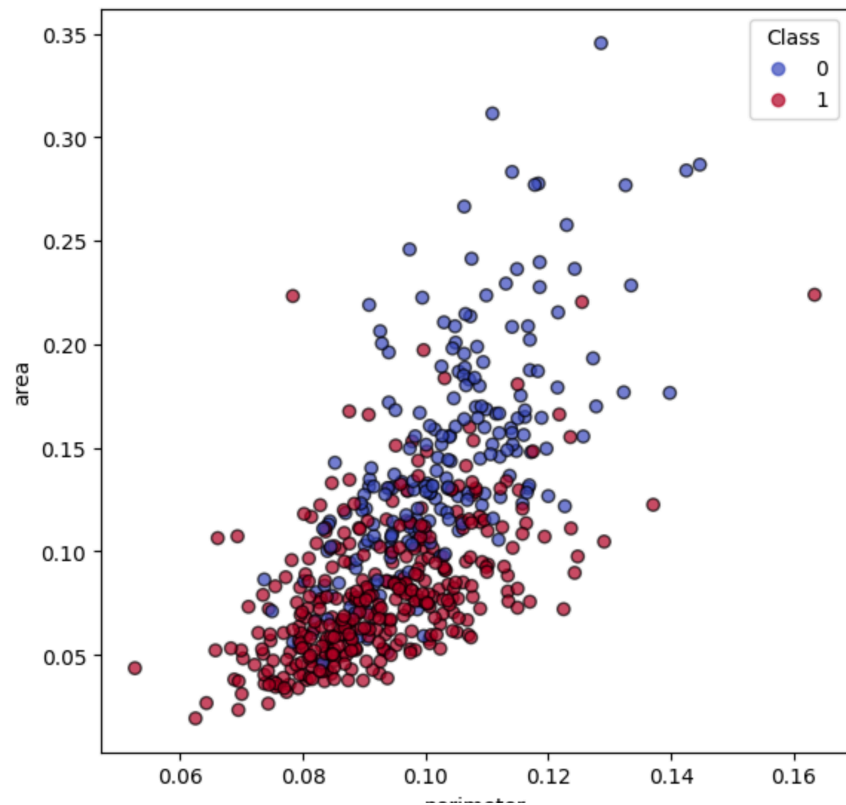
Setting: $\mu=0.0, \sigma=0.8$



Non-parametric density estimation

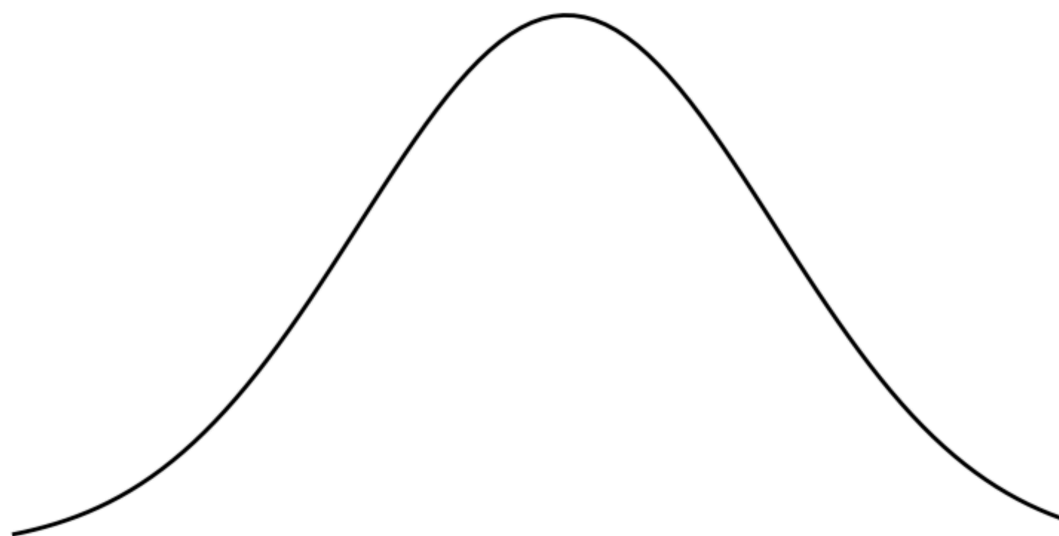
Overview

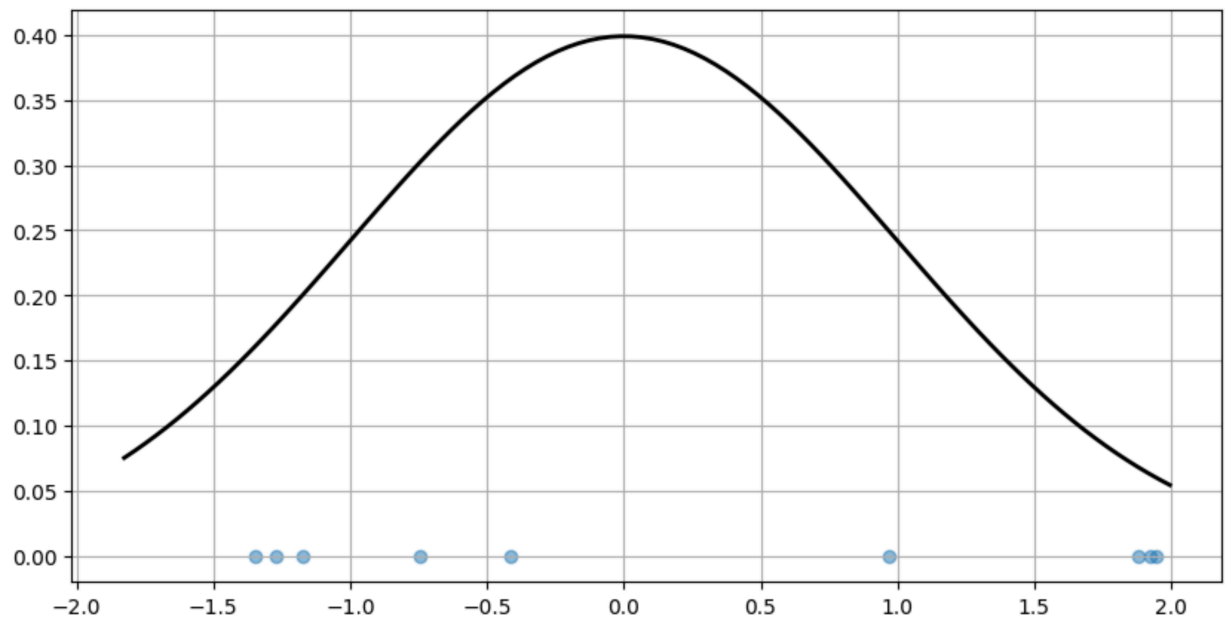
- Real-world data rarely follow exact distributions.
- So what do we do when we cannot find a parametric distribution that matches or data?
- Consider the following example



Histogram approach

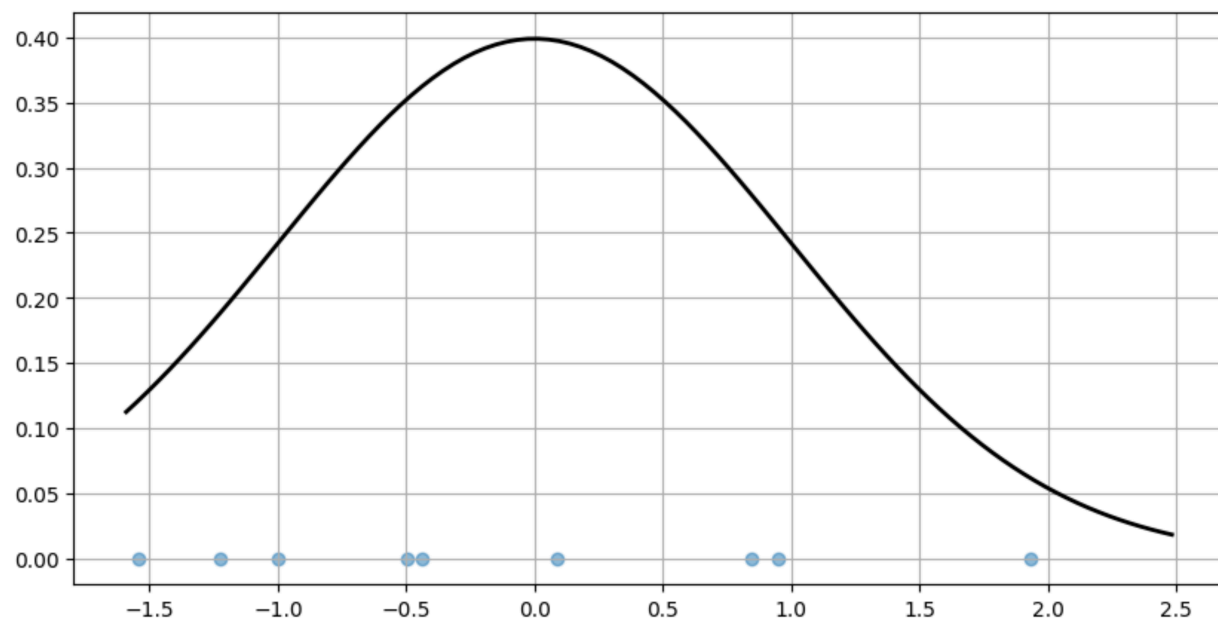
- Idea:
 - Divide input into bins.
 - For each bin; estimate probability of a sample x being in a bin.
 - This can probability can be estimated using the *frequency ratio*
 - $P \approx k_N/N$, where N is the total number of samples and k_N is the number of samples within the bin.





Alternative histogram approach

- Place bin on each sample.
- Count samples within bin.



Mathematical formulation of alternative approach

- Define

$$I(y) =$$

Parzen windows (Parzen, 1962)

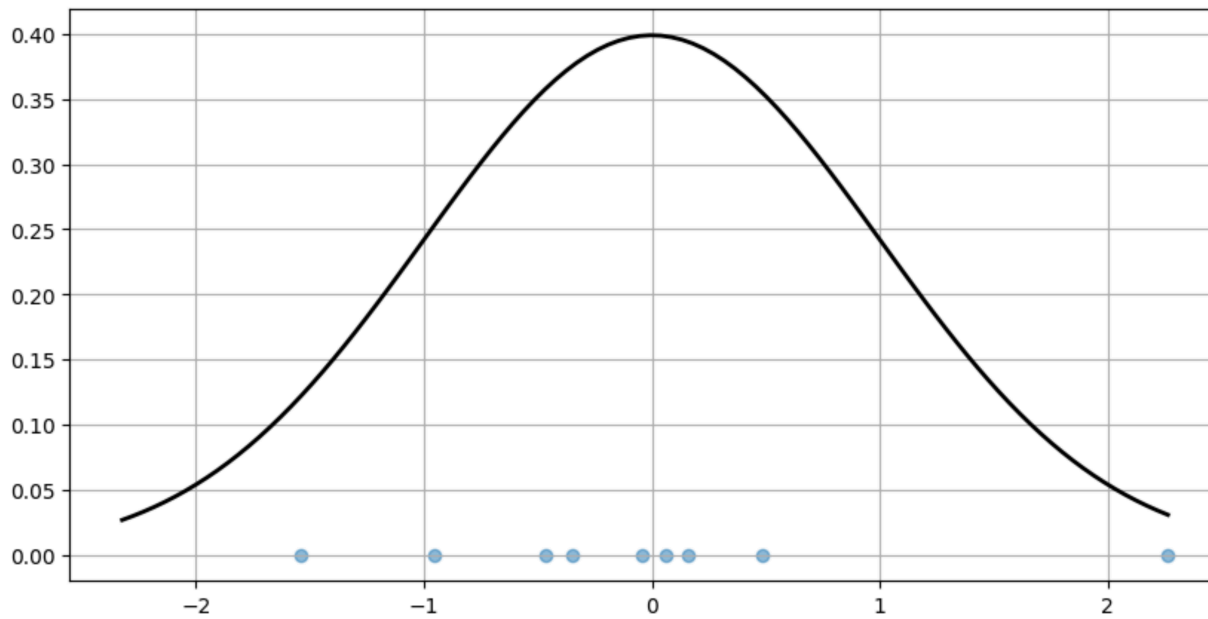
- Take the alternative histogram approach as a starting point.
- Replace I with a smooth function ϕ subject to:
 - $\phi(y) \geq 0$ and $\int \phi(y) dy = 1$
- This ensures an estimate that is a valid probability density function.
- A typical choice is $N(0, 1)$, which results in:

$$p(x) = \frac{1}{Nh} \sum_{i=1}^N \phi\left(\frac{x_i - x}{h}\right)$$

$$= 1 \text{ if } \frac{|x_i - x|}{h} \leq 1/2$$

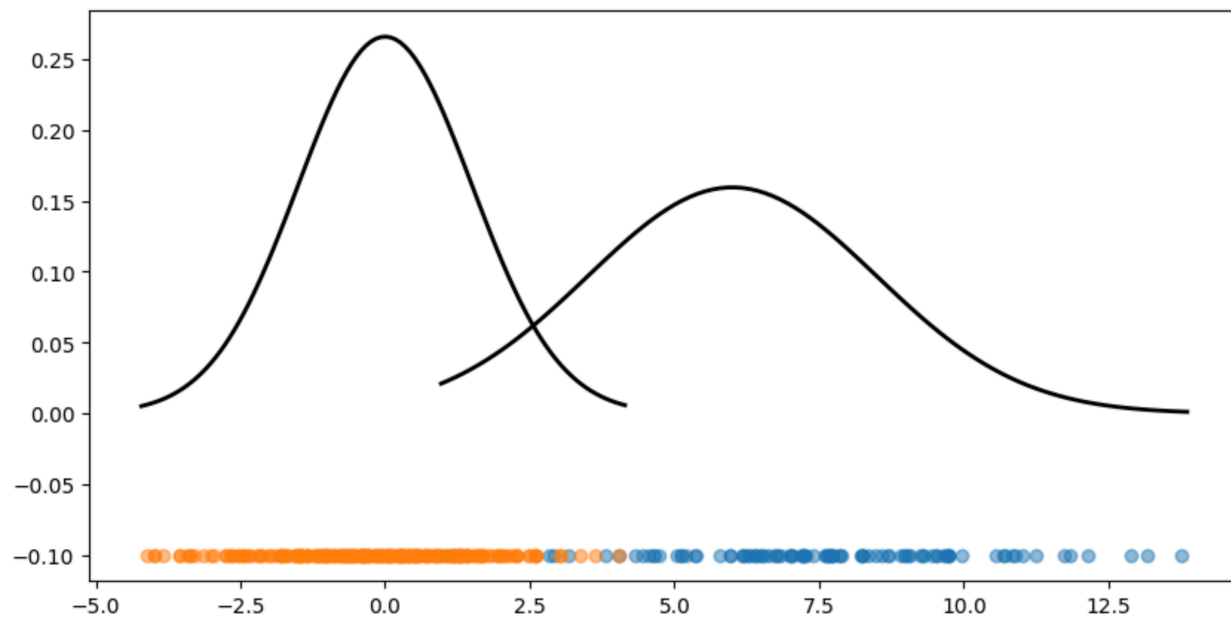
- Therefore:

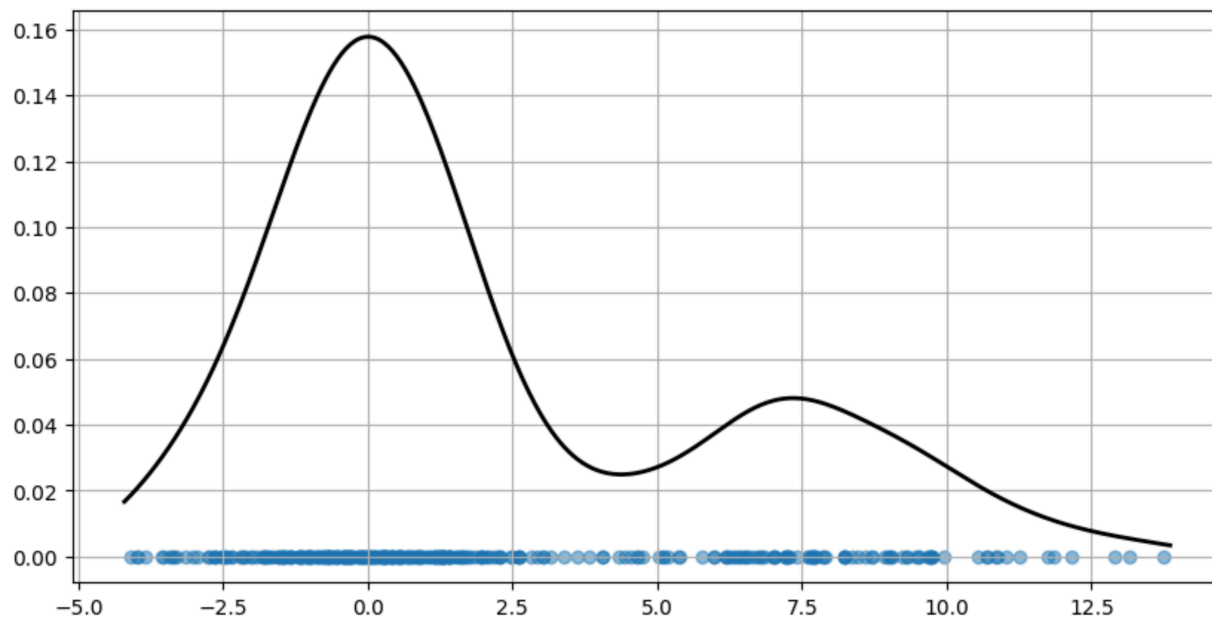
$$\begin{aligned} & \cdot p \\ & (x \\ &) \\ & \simeq \frac{1}{Nh} \\ & \sum_i^N I \end{aligned}$$



Parzen windows in practice

- Assume we have a fixed dataset with N samples.
- Then:
 - Bias low for "small" h .
 - Variance low for "large" h .
- Remarks:
 - Plug $\hat{p}(\mathbf{x}|w_i)$ into Bayes!
 - Need more data for higher dimensions to be accurate.





Parzen + Bayes: how does h shape the decision boundary?

- Now plug the Parzen estimate $\hat{p}(\mathbf{x}|w_i)$ into the Bayes rule:
 - Decide w_1 when $\hat{p}(\mathbf{x}|w_1) > \hat{p}(\mathbf{x}|w_2)$.
- The two classes below form the classic **two-moons** shape — each is a curved arc with noise, clearly not a Gaussian blob. A *parametric* Gaussian fit per class would draw a near-straight decision line through the overlap and misclassify the moons; Parzen windows with the right bandwidth recover the curved boundary.
- Click the tabs to see how the kernel width h trades bias vs. variance in the boundary.

☐ Show decision boundary

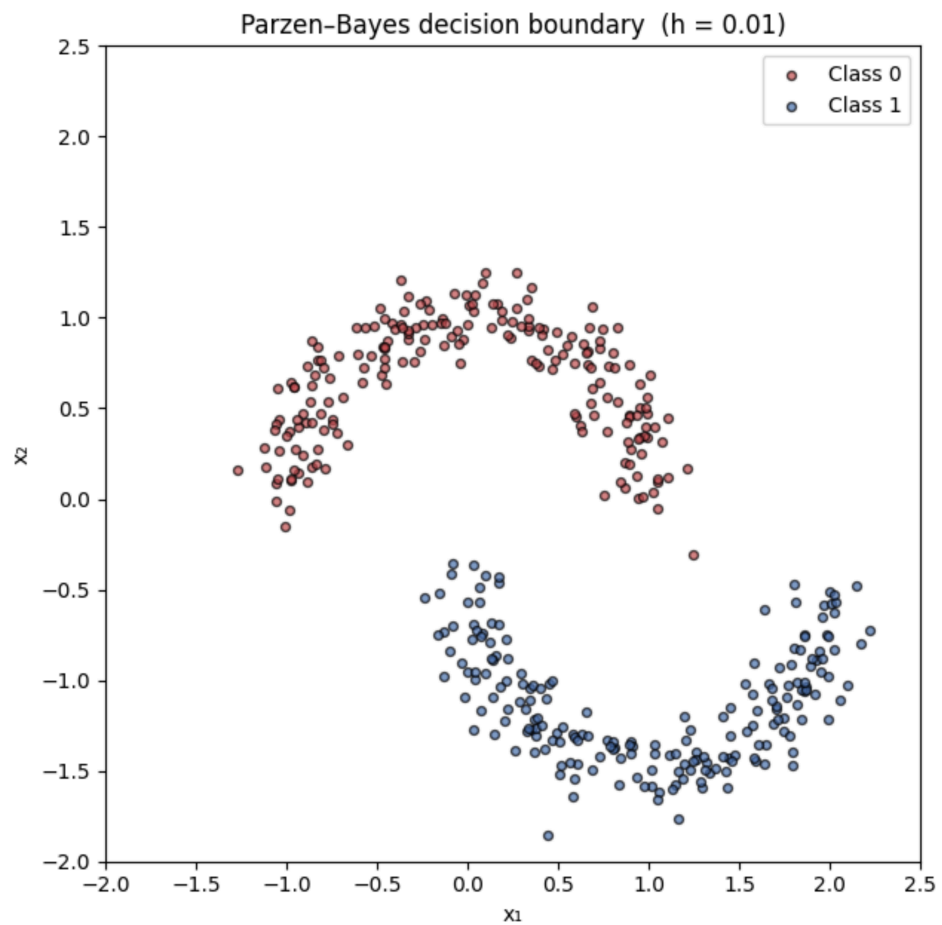
h = 0.01

h = 0.15

h = 0.50

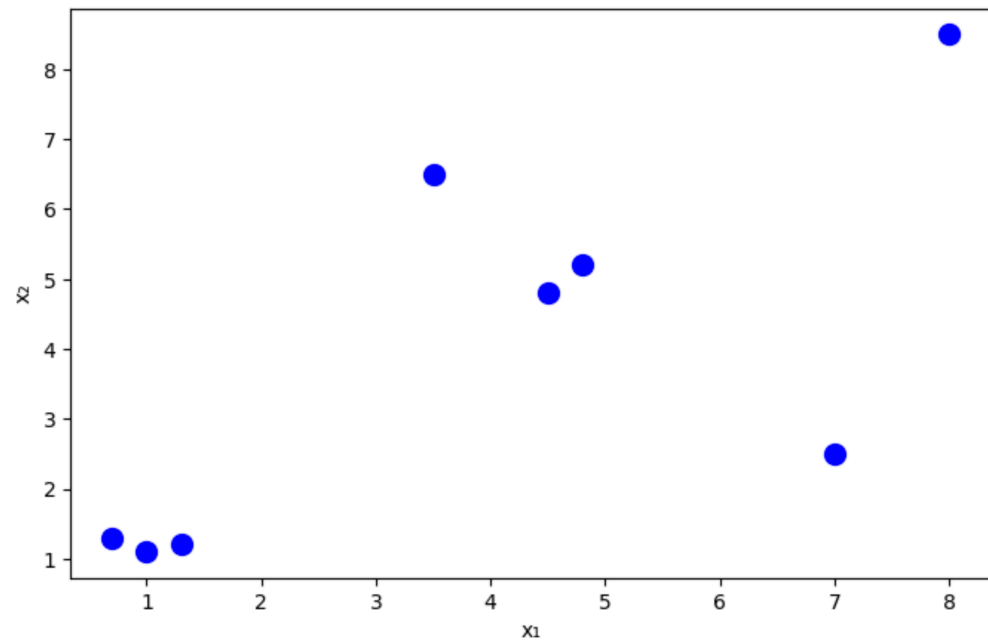
h = 1.50

Bandwidth: h = 0.01



k-Nearest neighbor density estimation

- Parzen estimation:
 - Points are fixed and put "area" around point.
- What happens if we reverse the roles?



k-Nearest neighbor classification

- Can create powerful classification algorithms using nearest neighbors.
- Simple idea:
 - Step 1:
 - Step 2:
 - Step 3:
- A bit simplified:
 - What can go wrong above?
 - Limitations?

Recent advances to address computational burden - Faiss

- Faiss is a library for efficient similarity search.
- [Github repository for Faiss](#).
- Example from [Pezzotti et al., 2020k](#)

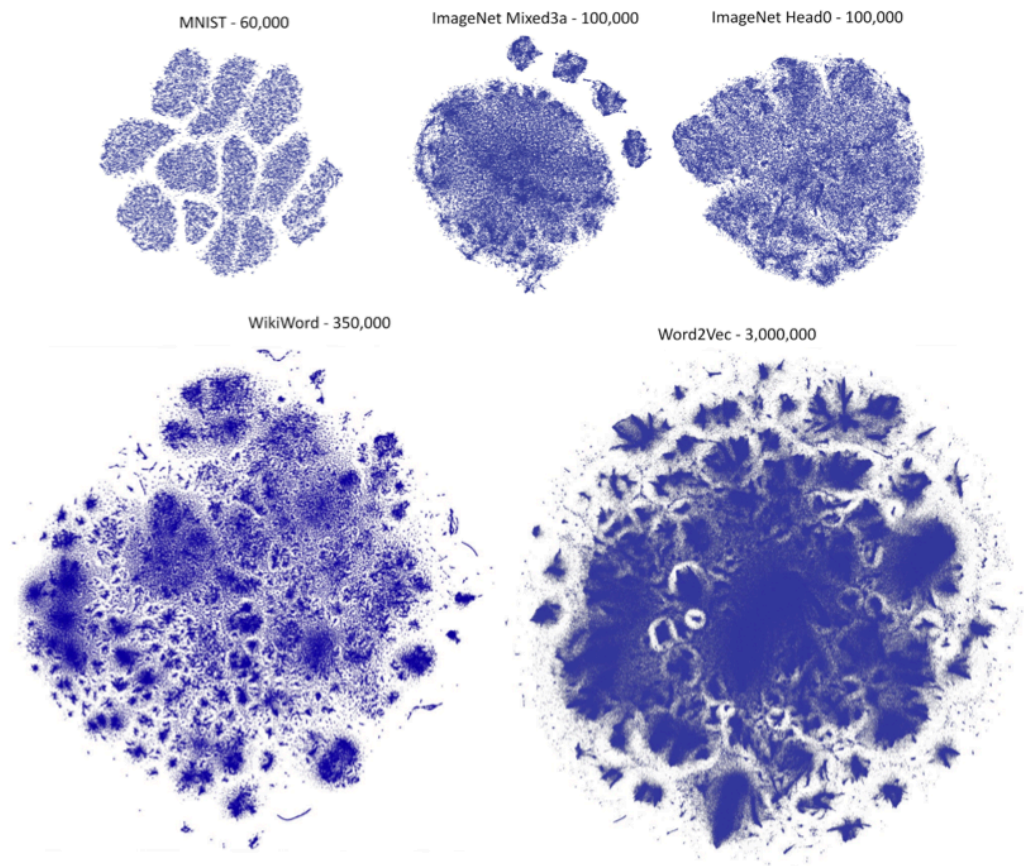


Fig. 5: **Embeddings** of the MNIST, ImageNet Mixed3a, ImageNet Head0, WikiWord and Word2Vec datasets generated by our technique.